

The image tag —

```

```

Self-closing. No `</img>`. Like `<br>` and `<hr>` — there is no content to wrap.

ATTRIBUTE	PURPOSE
src	Where the image file lives. Required.
alt	Text description. Required.
width	Size in pixels. Optional.
height	Size in pixels. Optional.

Both `src` and `alt` are required. Missing `alt` is the single most common accessibility bug in student work. Do not submit an image without it.

File paths — where the image lives

Relative paths — your own images

```
<!-- Same folder -->


<!-- Subfolder -->


<!-- Up one folder -->

```

Same idea as Lesson 4's relative URLs. Portable — move the folder, everything still works.

Absolute paths — other people's images

```
<!-- Full URL -->


<!-- Full file path -->

```

Breaks if you move the folder or share the project. Use for images hosted on other servers only.

Alt text — describe what it IS

```
<!-- Descriptive and specific -->


<!-- Context matters -->


<!-- Decorative only -->

```

Weak alt text

- `alt="image"` — says nothing
- `alt="dog"` — too vague
- `alt="photo.jpg"` — the filename is not a description
- `alt="image of a dog"` — the word “image” is redundant

Strong alt text describes what's happening, not just what it is. Same principle as Lesson 4's link text — write it to stand alone, out of context.

Image file formats

FORMAT	BEST FOR	PROS	CONS
.jpg	Photographs	Small file size, millions of colors	No transparency
.png	Logos, graphics, screenshots	Transparency, crisp edges	Larger file size
.gif	Simple animations	Supports animation	Limited to 256 colors

Rule of thumb. Photograph = JPG. Logo or graphic with hard edges or transparency = PNG. Animation = GIF. That covers 95% of student projects.

Sizing with HTML attributes

```
<!-- Both width and height -->


<!-- Just width — height auto-adjusts -->

```

Just width is safer. The height adjusts automatically and keeps the image's aspect ratio. Setting both can *distort* the image if the numbers are not proportional.

Never stretch a small image larger. It gets blurry. Down-sizing a large image works fine. HTML sizing is a preview — Unit 2 covers CSS sizing.

Placeholder image services

picsum.photos

Real Unsplash photos. Random by default.

```


<!-- locked-in specific photo -->

```

placeholder.co

Solid boxes with text labels.

```



```

placeholder.net

Themed art: avatars, products, maps.

```

```

Never use `via.placeholder.com`. The domain was sold in 2023 and the service went offline in 2024. Every URL returns a broken image. Old tutorials still reference it — ignore them.

A simple photo gallery

```
<body>
  <h1>My Photo Gallery</h1>

  <h2>Vacation</h2>
  
  
```

The gallery is heading + images, repeated. Everything here is from Lesson 3 (h1, h2, p, em) plus today's `<img>`. No new concepts — just composition.

Consistent sizing. Every image in the demo uses `width="300"`. Mixed widths make a gallery look unfinished.

```
<h2>Family</h2>


<p><em>All photos summer 2024</em></p>
</body>
```

The broken-image moment

Change `images/profile.jpg` to `images/profilee.jpg` (extra `e`) and save.

The browser shows a broken image icon, and the alt text appears in its place. That broken view is exactly what a screen reader user “sees” when alt text is missing.

Two things to look for when an image does not appear:

- Is the `src` path exactly right? (case, spelling, folder name)
- Is the image file actually in that folder?

Common mistakes to avoid

MISTAKE	WHAT GOES WRONG
Missing <code>alt</code>	Screen readers can't describe the image. Broken images show nothing useful.
Poor alt text — <code>alt="image"</code>	Technically present but useless out of context. Same as “click here” for links.
Wrong file path	Broken image icon. Check folder name, spelling, and case.
Stretching images	Setting both <code>width</code> and <code>height</code> to non-proportional numbers distorts the picture.
Closing <code></code>	There is no closing tag. <code></code> is self-closing like <code>
</code> .
Using <code>via.placeholder.com</code>	Dead service. Returns broken images. Use <code>picsum</code> , <code>placeholder.co</code> , or <code>placeholder.net</code> .

Today's syntax — quick reference

GOAL	CODE
Basic image	<code></code>
Same-folder image	<code></code>
Subfolder image	<code></code>
External image	<code></code>
Fixed size	<code>width="300" height="200"</code>
Auto-height	<code>width="300"</code>
Decorative image	<code>alt=""</code>

Before you submit — test every image

- Does each image appear in the browser?

- Does every `<img>` have a descriptive `alt` ?
- Are the sizes consistent within each section?
- Are the paths relative for your own images, absolute for others?
- Try the broken-image test: rename one file and see if the alt text appears in its place.

What is next. Your pages now have structure, text, links, lists, and images. Everything on this sheet is a variation of the same rule you have been following since Lesson 2 — every tag declares what something *is*, not what it looks like.

Keep this sheet. Every image you write from now on needs `src` , `alt` , and a path. Those three things never change.